

LABORATORY OBSERVATION/RECORD

Course:	Full Stack Web Development
Course Code:	23CM4121
Experiment No.:	3
Student Name:	P.Govindamma
Roll No.:	A24126552104
Date:	21-07-2026

1. TITLE

Write a Program Using JavaScript to Demonstrate Arrays, Functions, and Object-Oriented Programming (OOP) Concepts

2. AIM

To write and execute JavaScript programs that demonstrate the use of arrays and functions for numeric computation, and to demonstrate the four pillars of Object-Oriented Programming — encapsulation, inheritance, polymorphism, and abstraction — using JavaScript classes.

3. OBJECTIVE

- To understand how to define and call functions in JavaScript.
- To perform iteration over arrays using a for loop.
- To compute aggregate values (sum, average) from an array of numbers.
- To define classes using the class keyword.
- To implement encapsulation using private class fields (#field).
- To implement inheritance using the extends and super keywords.
- To implement polymorphism through method overriding.
- To implement abstraction by preventing direct instantiation of a base class.
- To use static properties shared across all instances of a class.
- To use array methods such as `forEach()`, `reduce()`, and `filter()` alongside class objects.

4. THEORY

JavaScript supports both procedural programming, using functions and arrays, and object-oriented programming, using the ES6 class syntax. This experiment combines both styles: a simple function-and-array program that computes the sum and average of a numeric array, and a more elaborate class-based program that models a vehicle rental system.

Functions and Arrays

A function is a reusable block of code that can accept parameters and return a value. An array stores an ordered collection of values, which can be traversed using a loop. In this experiment, `calculateSum()`

iterates through an array using a for loop to accumulate a total, and `calculateAverage()` reuses `calculateSum()` to compute the mean.

Classes and Objects

The `class` keyword defines a blueprint for creating objects with shared properties and methods. The `constructor()` method initializes new instances, and the `new` keyword creates an object from a class.

Encapsulation

Encapsulation hides internal state and exposes it only through controlled methods. In the `Vehicle` class, the `#odometer` field is private (prefixed with `#`) and cannot be accessed directly from outside the class. It can only be read through the mileage getter and modified through the `drive()` method.

Inheritance

Inheritance allows a class to reuse the properties and methods of another class. `Car` and `Bike` both use `extends Vehicle`, and call `super(brand, model, ratePerDay)` inside their constructors to initialize the inherited fields defined in `Vehicle`.

Polymorphism

Polymorphism allows different classes to provide their own implementation of the same method name. Both `Car` and `Bike` override the `describe()` method to return a description specific to their own properties, and `Car` also overrides `calculateCost()` to add an insurance fee while still reusing the parent's logic via `super.calculateCost(days)`.

Abstraction

Abstraction hides implementation detail behind a simple interface. The `Vehicle` class is designed to be abstract: its constructor checks `new.target === Vehicle` and throws an error if someone tries to create a `Vehicle` object directly, so only meaningful subclasses like `Car` and `Bike` can be instantiated. Its `describe()` method also throws an error by default, forcing every subclass to supply its own implementation.

Static Properties

The `static` keyword defines a property that belongs to the class itself rather than to any individual instance. `Vehicle.totalVehicles` is incremented in the constructor and tracks how many vehicles have been created in total, across every subclass.

The basic flow of the application is:

Function/Array Program: Array Input → Loop-based Sum → Reused Average Function → Console Output.

OOP Program: Class Definitions (`Vehicle`, `Car`, `Bike`) → Fleet Creation → Method Calls (`describe`, `rent`, `drive`, `calculateCost`) → Array Processing (`reduce`, `filter`) → Console Output

5. ALGORITHM / PROCEDURE

- 1 Part A — Functions and Arrays
- 2 Define a function `calculateSum(numbers)` that initializes a sum to zero.
- 3 Use a for loop to iterate through each element of the array and add it to sum.
- 4 Return the accumulated sum from the function.
- 5 Define a function `calculateAverage(numbers)` that calls `calculateSum()` and divides the result by the array length.
- 6 Create an array of numbers.
- 7 Call `calculateSum()` and `calculateAverage()` on the array and store the results.

8 Print the array, sum, and average to the console.

10⁹Part B — Object-Oriented Programming

11 Define an abstract base class `Vehicle` with a static `totalVehicles` property and a private `#odometer` field.

12 In the constructor, check `new.target` to prevent direct instantiation of `Vehicle`, and initialize `brand`, `model`, `ratePerDay`, and `isRented`.

13 Add a mileage getter to read the private `#odometer` field.

14 Add a `drive(km)` method that validates the input and updates `#odometer`.

15 Add `rent()` and `returnVehicle()` methods to toggle the `isRented` flag.

16 Add a `describe()` method in the base class that throws an error, to be overridden by subclasses.

17 Add a `calculateCost(days)` method that multiplies `ratePerDay` by `days`.

18 Define a `Car` class that extends `Vehicle`, calling `super()` in its constructor and adding a `seats` property.

19 Override `describe()` in `Car` to include the number of seats.

20 Override `calculateCost()` in `Car` to add a daily insurance fee on top of the base cost from `super.calculateCost(days)`.

21 Define a `Bike` class that extends `Vehicle`, calling `super()` in its constructor and adding a `hasGears` property.

22 Override `describe()` in `Bike` to include the gear information, without overriding `calculateCost()`.

23 Write a `createFleet()` function that returns an array of `Car` and `Bike` instances.

24 Write a `totalFleetRevenue(fleet, days)` function using `reduce()` to sum the cost of every vehicle in the fleet.

25 Write a `filterByType(fleet, VehicleClass)` function using `filter()` and `instanceof` to select vehicles of one subclass.

26 Create the fleet, call `describe()` on every vehicle, and print the results.

27 Rent and drive one vehicle, read its mileage through the getter, and return it.

28 Calculate and print the 3-day cost for every vehicle, and the total fleet revenue.

29 Filter and print only the `Car` instances from the fleet.

30 Print the value of the static `Vehicle.totalVehicles` property.

31 Attempt to instantiate `Vehicle` directly inside a `try/catch` block and print the caught error message.

6. PROGRAM

Part A — arrays.js (Functions and Arrays)

```
function calculateSum(numbers) {
  let sum = 0;

  for (let i = 0; i < numbers.length; i++) {
    sum = sum + numbers[i];
  }

  return sum;
}

// Function to calculate average
function calculateAverage(numbers) {
  let sum = calculateSum(numbers);
```

```

    return sum / numbers.length;
}

// Array of numbers
let numbers = [10, 20, 30, 40, 50];

// Calling functions
let sum = calculateSum(numbers);
let average = calculateAverage(numbers);

// Display results
console.log("Array Elements:", numbers);
console.log("Sum:", sum);
console.log("Average:", average);

```

Part B — oop.js (OOP: Encapsulation, Inheritance, Polymorphism, Abstraction)

```

class Vehicle {
  // Static property: shared across ALL instances of Vehicle (and subclasses)
  static totalVehicles = 0;

  // ENCAPSULATION: private fields (marked with #) can't be accessed
  // directly from outside the class - only through methods we define
  #odometer = 0;

  constructor(brand, model, ratePerDay) {
    if (new.target === Vehicle) {
      // Prevents anyone from creating a plain "Vehicle" directly -
      // it's meant to be abstract (only subclasses should be instantiated)
      throw new Error("Vehicle is abstract and cannot be instantiated directly");
    }
    this.brand = brand;
    this.model = model;
    this.ratePerDay = ratePerDay;
    this.isRented = false;
    Vehicle.totalVehicles++;
  }

  // Getter: reads the private #odometer field in a controlled way
  get mileage() {
    return this.#odometer;
  }

  // Encapsulated method: the only way to change #odometer from outside
  drive(km) {
    if (km < 0) {
      console.log("Distance can't be negative.");
      return;
    }
    this.#odometer += km;
  }

  rent() {
    this.isRented = true;
    console.log(`${this.brand} ${this.model} has been rented.`);
  }

  returnVehicle() {
    this.isRented = false;
    console.log(`${this.brand} ${this.model} has been returned.`);
  }

  // ABSTRACTION: subclasses are expected to override this.
  // The base version throws an error if a subclass forgets to implement it.
  describe() {
    throw new Error("describe() must be implemented by subclass");
  }

  // Regular method available to all subclasses via inheritance
  calculateCost(days) {
    return this.ratePerDay * days;
  }
}
// -----

```

```

// INHERITANCE: Car and Bike both extend Vehicle,
// reusing its properties and methods
// -----
class Car extends Vehicle {
  constructor(brand, model, ratePerDay, seats) {
    super(brand, model, ratePerDay); // calls Vehicle's constructor
    this.seats = seats;
  }

  // POLYMORPHISM: Car provides its own version of describe()
  describe() {
    return `${this.brand} ${this.model} (Car) - ${this.seats} seats - $$${this.ratePerDay}/day`;
  }

  // POLYMORPHISM: Car overrides calculateCost with extra logic,
  // but still reuses the parent's version via super
  calculateCost(days) {
    const baseCost = super.calculateCost(days);
    const insurance = days * 5; // cars have a daily insurance fee
    return baseCost + insurance;
  }
}

class Bike extends Vehicle {
  constructor(brand, model, ratePerDay, hasGears) {
    super(brand, model, ratePerDay);
    this.hasGears = hasGears;
  }

  // POLYMORPHISM: Bike's own version of describe()
  describe() {
    return `${this.brand} ${this.model} (Bike) - Gears: ${this.hasGears} - $$${this.ratePerDay}/day`;
  }
  // No override of calculateCost() - Bike just uses Vehicle's version directly
}

// -----
// Regular functions (not tied to a class)
// working alongside the OOP structure
// -----

// Function: create and return a fleet (array) of vehicles
function createFleet() {
  return [
    new Car("Toyota", "Camry", 40, 5),
    new Car("Honda", "Civic", 35, 5),
    new Bike("Yamaha", "FZ", 15, true),
    new Bike("Hero", "Splendor", 10, false),
  ];
}

// Function: total potential rental income for a fleet over N days
function totalFleetRevenue(fleet, days) {
  return fleet.reduce((sum, vehicle) => sum + vehicle.calculateCost(days), 0);
}

// Function: filter only vehicles matching a type (using instanceof)
function filterByType(fleet, VehicleClass) {
  return fleet.filter((vehicle) => vehicle instanceof VehicleClass);
}

// --- Program execution ---

const fleet = createFleet();

console.log("--- Fleet Descriptions ---");
fleet.forEach((v) => console.log(v.describe())); // polymorphism in action

console.log("\n--- Renting & Driving ---");
fleet[0].rent();
fleet[0].drive(120);
fleet[0].drive(80);
console.log(`${fleet[0].brand}'s mileage: ${fleet[0].mileage} km`); // uses getter
fleet[0].returnVehicle();

```

```

console.log("\n--- Cost for 3 days rental ---");
fleet.forEach((v) =>
  console.log(`${v.brand} ${v.model}: ${v.calculateCost(3)}`)
);

console.log("\n--- Total Fleet Revenue (3 days) ---");
console.log(`${totalFleetRevenue(fleet, 3)}`);

console.log("\n--- Only Cars ---");
filterByType(fleet, Car).forEach((c) => console.log(c.describe()));

console.log("\n--- Total Vehicles Created (static property) ---");
console.log(Vehicle.totalVehicles);

console.log("\n--- Trying to instantiate abstract class directly ---");
try {
  const v = new Vehicle("Generic", "X", 10);
} catch (err) {
  console.log("Error caught:", err.message);
}

```

7. CODE EXPLANATION

Code	Explanation
for loop	Iterates through each element of the numbers array to accumulate a total.
function name(params) {...}	Defines a reusable, named block of code that can be called with arguments.
return	Sends a value back from a function to the code that called it.
class	Defines a blueprint for creating objects with shared structure and behaviour.
constructor()	Special method that runs automatically when a new object is created.
#fieldName	Declares a private class field, accessible only from within the class.
get propertyName()	Defines a getter that lets a private field be read like a normal property.
static property	Defines a property shared by the class itself, not by individual instances.
extends	Allows one class to inherit the properties and methods of another class.
super(...)	Calls the parent class's constructor or method from within a subclass.
new.target	Refers to the class that new was called on; used here to block direct Vehicle instantiation.
instanceof	Checks whether an object was created from a particular class.
forEach()	Executes a given function once for every element in an array.
reduce()	Combines all array elements into a single accumulated value.
filter()	Creates a new array containing only elements that satisfy a condition.
try / catch	Runs code that might throw an error and handles that error gracefully.

8. TEST CASES AND EXECUTION DATA

Test Case	Input / Action	Expected Result	Actual Result	Status
TC-01	Run arrays.js with [10, 20, 30, 40, 50]	Sum = 150, Average = 30	Sum: 150, Average: 30	Pass

Test Case	Input / Action	Expected Result	Actual Result	Status
TC-02	Call describe() on each vehicle in the fleet	Each vehicle prints its own type-specific description	Correct Car/Bike descriptions printed	Pass
TC-03	Rent, drive (120 + 80 km), then return the first vehicle	Mileage should read 200 km via the getter	Toyota's mileage: 200 km	Pass
TC-04	Call drive() with a negative number	Should reject the update and print a warning	"Distance can't be negative." printed	Pass
TC-05	Calculate 3-day cost for each vehicle	Cars include a \$15 insurance fee (3 x \$5); Bikes do not	Toyota: \$135, Honda: \$120, Yamaha: \$45, Hero: \$30	Pass
TC-06	Calculate total fleet revenue for 3 days	Sum of all four vehicles' 3-day costs	\$330	Pass
TC-07	Filter fleet by Car using instanceof	Only the two Car objects should be returned	Toyota Camry and Honda Civic listed	Pass
TC-08	Check Vehicle.totalVehicles after creating the fleet	Should equal the number of vehicles created	4	Pass
TC-09	Attempt new Vehicle("Generic", "X", 10) directly	Should throw an error caught by try/catch	"Vehicle is abstract and cannot be instantiated directly"	Pass

9. OUTPUT

Output 1: arrays.js (node arrays.js)

```

Array Elements: [ 10, 20, 30, 40, 50 ]
Sum: 150
Average: 30

```

Output 2: oop.js (node oop.js)

```

--- Fleet Descriptions ---
Toyota Camry (Car) - 5 seats - $40/day
Honda Civic (Car) - 5 seats - $35/day
Yamaha FZ (Bike) - Gears: true - $15/day
Hero Splendor (Bike) - Gears: false - $10/day

--- Renting & Driving ---
Toyota Camry has been rented.
Toyota's mileage: 200 km
Toyota Camry has been returned.

--- Cost for 3 days rental ---
Toyota Camry: $135
Honda Civic: $120
Yamaha FZ: $45
Hero Splendor: $30
--- Total Fleet Revenue (3 days) ---

```

\$330

```
--- Only Cars ---  
Toyota Camry (Car) - 5 seats - $40/day  
Honda Civic (Car) - 5 seats - $35/day  
  
--- Total Vehicles Created (static property) ---  
4  
  
--- Trying to instantiate abstract class directly ---  
Error caught: Vehicle is abstract and cannot be instantiated directly
```

10. OBSERVATION / INTERVIEW QUESTIONS

Q1. What is the difference between a function declaration and calling a function?

Answer: A function declaration (function calculateSum(numbers) {...}) defines the reusable block of logic once. Calling the function, such as calculateSum(numbers), executes that logic with specific argument values and can return a result.

Q2. What is encapsulation, and how is it implemented in the Vehicle class?

Answer: Encapsulation means hiding internal state and only exposing it through controlled methods. The Vehicle class declares #odometer as a private field, which cannot be accessed directly from outside the class; it can only be read through the mileage getter or updated through the drive() method.

Q3. How does inheritance work between Vehicle, Car, and Bike?

Answer: Car and Bike both use extends Vehicle, which lets them reuse Vehicle's properties and methods. Each subclass calls super(brand, model, ratePerDay) in its own constructor to run Vehicle's constructor logic before adding its own properties (seats or hasGears).

Q4. What is polymorphism, and where is it demonstrated in this program?

Answer: Polymorphism allows different classes to respond differently to the same method call. Both Car and Bike override describe() to return a class-specific description, and calling v.describe() on each fleet vehicle produces different output depending on its actual type.

Q5. How is abstraction achieved in the Vehicle class?

Answer: The Vehicle constructor checks new.target === Vehicle and throws an error if someone tries to instantiate Vehicle directly, forcing it to only be used through subclasses. Its describe() method also throws an error by default, requiring every subclass to provide its own implementation.

Q6. What is the purpose of the static totalVehicles property?

Answer: static totalVehicles belongs to the Vehicle class itself rather than to any single instance. It is incremented once in the constructor every time a new vehicle (Car or Bike) is created, so it tracks the total count across the entire fleet.

Q7. What do reduce() and filter() do in the totalFleetRevenue() and filterByType() functions?

Answer: reduce() combines every vehicle's calculateCost(days) into a single accumulated total, giving the fleet's overall revenue. filter() uses instanceof to build a new array containing only the vehicles that match a given class, such as Car.

11. RESULT

Thus, JavaScript programs demonstrating arrays, functions, and Object-Oriented Programming concepts were successfully written and executed. The array-and-function program correctly computed

the sum and average of a numeric array, and the OOP program successfully demonstrated encapsulation using private fields, inheritance using extends and super, polymorphism through method overriding, and abstraction through a non-instantiable base class, along with static properties and array methods such as `forEach()`, `reduce()`, and `filter()`. All outputs were verified against the expected results.